RTOS Görev Planlayıcısı ve Zamanlama Analizi

FreeRTOS üzerinde 3 eş zamanlı görev yapmam gerekiyodu.

1-Sensör Okuma : MPU6050 sensörün sadece accel_z eksenindeki ham verisi üzerinden task'ı planladım.

2- Veri paketleme : Ayrı bir task olarak daha düşük öncelikte z ekseninden alınan veriyi 8 baytlık bir paket haline getirdim.

3-UART çıkışı : Düşük öncelik olarak ayarladığım bu son task'ta paketlenen 8 baytlık veri queue mantığında devreye girerek UART protokolü ile veri iletimini sağlıyor.

Bu ödevde STM32 mikrodenetleyicisi üzerinde ThreadX RTOS'unu tercih ettim. Kartta hazır yükleme yapmadan kullanılabilmesi ve nispeten daha basit makroları sayesinde işlem kolaylığı sağladı bana.

Thread_1_sensor (Priority: 5 - Yüksek):

Thread_2_data_pack (Priority: 10 - Orta):

Thread_3_UART (Priority: 15 - Düşük):

Şeklinde ayarlanmıştır.

Expression	Type	Value
counter_1	volatile uint8_t	150 '\226
counter_2	volatile uint8_t	150 '\226
counter_3	volatile uint8_t	150 '\226
accel_z	volatile int16_t	-16036
temp_raw	volatile int16_t	3232
paket	UART_Packet_t	{...}
data	uint8_t [5]	[5]
data[0]	uint8_t	170 '\252
data[1]	uint8_t	85 'U'
data[2]	uint8_t	193 'Á'
data[3]	uint8_t	92 '\\'
data[4]	uint8_t	10 '\n'
padding	uint8_t [3]	[3]
padding[0]	uint8_t	0 '\000'
padding[1]	uint8_t	0 '\000'
padding[2]	uint8_t	0 '\000'
Add new expression		

Burda masaya ters olarak yerleştirilmiş mpu6050 sensörünü tam düz şekilde tutmaya çalışarak aldığım verileri görüyoruz. Cube ide'den debug edip aldığım sonuçlar bunlar.

Counter_x olarak tanımladığım değişkenler her bir taskın içinde task her bir döngüye girdiğinde değerini 1 artırarak bana taskların çalıştığını ve her taskın birbirini takip ettiğini anlamamı sağlıyor. Burda değerleri 150 olarak gözüküyor dolayısıyla 150 kere RTOS üzerinden bu tasklar çalışmış ve başarı ile sonlanmış.

Accel_z değişkenim ise mpu sensöründen alınan ham veriyi ifade ediyor. Masaya ters tuttuğum için (–) alan bu değer datasheetteki 1g olan 16384 değerine çok yakın dolayısıyla sensörden de sağlıklı olabilecek bir veri geldiğini burdan anlayabiliriz ama tabiki ileriki süreçlerde açı değerlerine dönüştürülüp filtreleme işlemlerinden sonra sağlıklı olduğuna karar vermemiz gerekiyor. Şuan sadece bize veri sağlaması bu görev için yeterli olacaktır.

Temp_raw mpu sensor iç sıcaklığını ölçüp bize aktaran bir data yine.

Paket olarak tanımladığım yapı ise veri paketlerinin içeriğini bize gösteriyo. Veri paketleme aşamasında 8 bayt olarak açılan yerde aslında bizim verimiz sadece 5 bayt büyüklüğünde burda değerleri görebilirsiniz data[0] ve data[1] header data[2], data[3] bizim sensor verilerimizin oluşturduğu kısım ve data[4] yeni satır (“\n”) şeklinde. Kalan kısımlar ise padding ile dolgu işlemi yapılmış boş baytlardır.

Burda özellikle kod yazarken belirli aşamalara dikkat ettim veriyi daha optimize bir şekilde osiloskopta görüntüleyebilmek için. Öncelikle ThreadX içinde kendi bulunan tx_thread_sleep() komutundan kaçındım ve kendi basit sleep komutumla periyot zamanının daha ince hassasiyetle kontrolünü sağladım.

```
ULONG anlik_zaman = tx_time_get();
if (sonraki_zaman>anlik_zaman){

    tx_thread_sleep(sonraki_zaman-anlik_zaman);

}
sonraki_zaman += Task_period;

}
```

Bu sayede 10ms olan birinci taskımızın olası gecikmelerde 10ms boyunca boş uyumak yerine süreyi kısaltıp sürekli 10ms döngüsünde kalmasını sağladım.

Sonrasında sadece Task_1 için oluşturduğum bu periyodu diğer tasklarda Queue (kuyruk) mantığında bu işlem bittikten sonra sırasıyla diğerlerininde vakit kaybetmeden

işlemlerini hızlıca yapmasını sağladım. Bu sayede kuyruk boşken bu tasklar işlemcide hiç güç harcamadan tamamen uykuda bekliyor.

İncelemek isterseniz tüm kodu raporun en altına ekliyorum.

Görev gecikmeleri ve Osiloskop ölçümleri:

Kartı sonrasında fiziksel ortamda test etmek için yükleyip osiloskop ölçümüne geldik. Bu ölçümde fotoğrafta görüldüğü gibi öncelikle WCET(Worst case execution time) ölçümü aldık. Görselde gözüken channel'lar sırasıyla

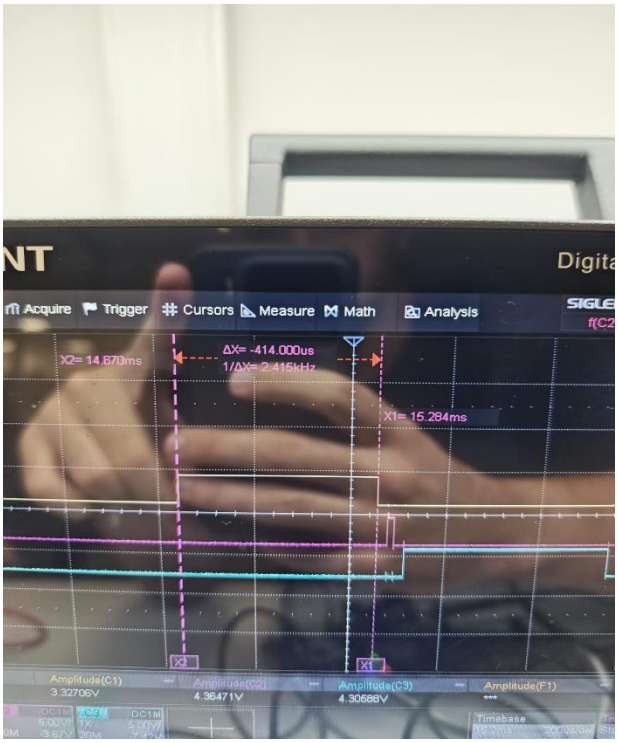
CH1- Task_1_Sensor

CH2- Task_2_Data_pack

CH3- Task_3_UART

Olarak seçilmiştir WCET analizimizi sensor üzerinden alıyoruz çünkü prioritysi yüksek olan ve olayı yöneten sinyalimiz sensor verisidir.

WCET: İşlemcinin bir görevi bitirmesi için gereken maksimum süredir. Neden önemli çünkü biz bu görevde periyodu 10ms olarak seçtik eğer execution time örneğin 12 ms sürseydi bir tane periyodu arada kaçırmış olurduk ve tamamen tüm verilerin hatalı gelmesine yol açardı bu durum. Benim yazmış olduğum kod ile işlemci bu sensor verisi işlemini sadece 0.4 ms de tamamlıyo ve diğer tasklara geçiyo. Geriye kalan diğer vakitlerinde güç harcamadan uykuda bekliyo.



Jitter: Görevin her döngüde aynı aralıkta çalışıp çalışmadığıdır. Benim ölçümlerimde jitter 1000ns yi geçmedi yani 0.01ms civarlarında olduğunu söyleyebiliriz güzel ve sabit bir değer.

Neden önemli?

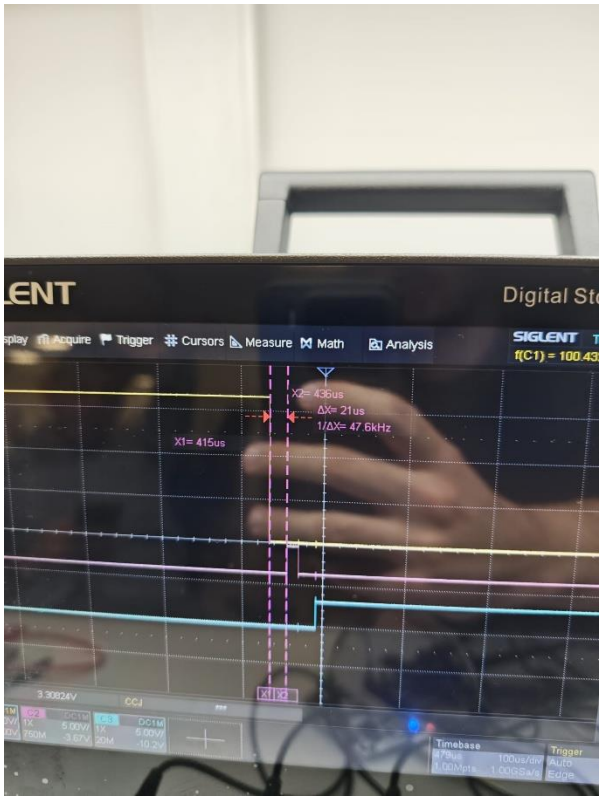
MPU6050'den ivme ve jiroskop okuyoruz ilerdeki ödevimde bu iki veriyi birleştirip pitch/roll tahminleri yapabilmem için filtreler kullanıcam bu gibi filtrelerin içindeki matematik formüllerinde zamana bağlı türev ve integraller bulunur. Burda sensörün tam olarak 10.000ms de bir okuduğunu varsayarak hesaplama yapıcaktı eğer + - olarak jitter daha yüksek bir değerde olsaydı formüldeki zamana bağlı ifadeler drifte uğrar ve filtre hatalı çalışırdı.

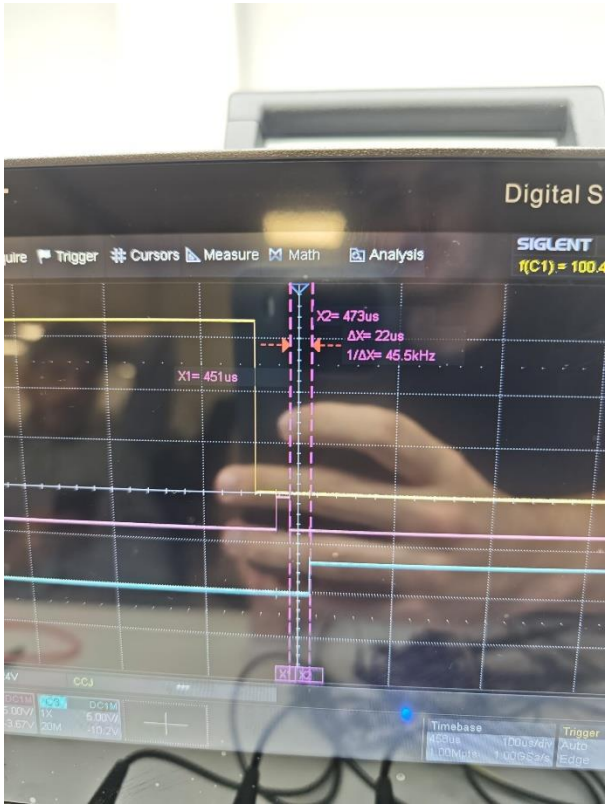
Jitter Ölçüm Videosu:

https://drive.google.com/file/d/1rkGO8Ct-mB45-uS74MhOnanLSKxfucnY/view?usp=sharing

Genel dalga formu videosu:
https://drive.google.com/file/d/1ohfpVHJQClDfoDilY1EsCXGRv0EgqPN0/view?usp=sharing

Burda da diğer ölçüm aralıkları:





```c
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file    app_threadx.c
  * @author  MCD Application Team
  * @brief   ThreadX applicative file
  ******************************************************************************
    * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Includes ------------------------------------------------------------------*/
#include "app_threadx.h"
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
extern I2C_HandleTypeDef hi2c1;
extern UART_HandleTypeDef huart2;
/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
//AD0 is GND so MPU6050 I2C address is 0x68
#define MPU6050_ADDR (0x68<<1)
#define MPU6050_REG_PWR_MGMT_1 0x6B
#define MPU6050_REG_ACCEL_XOUT_H 0x3B

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN PV */
TX_THREAD my_thread_sensor; //1. task'ın ID'si
TX_THREAD my_thread_data_pack; //2. task'ın ID'si
TX_THREAD my_thread_UART; //3. Task'ın ID'si

volatile uint8_t counter_1 = 0;
volatile uint8_t counter_2 = 0;
volatile uint8_t counter_3 = 0;
```

```
TX_QUEUE sensor_queue;
TX_QUEUE uart_queue;

uint8_t thread_1_stack[512]; //1.Task'ın  (512 byte RAM)
uint8_t thread_2_stack[512]; // 2. Task'ın (512 byte RAM)
uint8_t thread_3_stack[512]; //3. Task'ın (512 byte RAM)

ULONG sensor_queue_memory[10*1]; //1 ULONG 4 byte x 10 mesaj sensor Z verisi için
ULONG uart_queue_memory[10*2]; //2 ULONG 8byte x 10 mesaj uart data pack için

typedef struct {

       uint8_t data[5]; //5 byte lık veri paketimiz  (AA 55 Z_High Z_Low \n);
       uint8_t padding[3]; //8 byte doldurmak için padding
} UART_Packet_t;

UART_Packet_t paket;
uint8_t uart_tx_buffer[16];

//RAW DATA FROM MPU
volatile int16_t accel_x, accel_y , accel_z;
volatile uint16_t gyro_x , gyro_y, gyro_z;
volatile int16_t temp_raw;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
/* USER CODE BEGIN PFP */
void Thread_1_sensor(ULONG thread_input);
void Thread_2_data_pack(ULONG thread_input);
void Thread_3_UART(ULONG thread_input);

/* USER CODE END PFP */

/**
  * @brief  Application ThreadX Initialization.
  * @param memory_ptr: memory pointer
  * @retval int
  */
UINT App_ThreadX_Init(VOID *memory_ptr)
{
  UINT ret = TX_SUCCESS;
  /* USER CODE BEGIN App_ThreadX_MEM_POOL */

  /* USER CODE END App_ThreadX_MEM_POOL */
  /* USER CODE BEGIN App_ThreadX_Init */

  uint8_t pwr_cmd = 0x00;
  HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, MPU6050_REG_PWR_MGMT_1,
                        I2C_MEMADD_SIZE_8BIT, &pwr_cmd, 1, 100);

  //Task'ı Schedular'a save  (priority: 5 - HIGH)
  tx_thread_create(&my_thread_sensor,"Sensor Thread ", Thread_1_sensor,
                           0,thread_1_stack , sizeof(thread_1_stack)
                                    ,5 , 5,TX_NO_TIME_SLICE,TX_AUTO_START);
  //Task'ı Schedular'a save (Priority : 10 - Middle )
  tx_thread_create(&my_thread_data_pack, "Data Package Thread", Thread_2_data_pack, 0,
                     thread_2_stack, sizeof(thread_2_stack),
                     10, 10, TX_NO_TIME_SLICE, TX_AUTO_START);
  //TASK Schedular'a save (Priority : 15 LOW)
```

```
  tx_thread_create(&my_thread_UART , "UART Thread ", Thread_3_UART,0 ,
                                     thread_3_stack,sizeof(thread_3_stack)
                                     ,15,15,TX_NO_TIME_SLICE,TX_AUTO_START );
  //QUEUE'yi Schedular'a save
  tx_queue_create(&sensor_queue, "Sensor Queue", TX_1_ULONG,
                  sensor_queue_memory, sizeof(sensor_queue_memory));
  //UART'i Schedular'a save
  tx_queue_create(&uart_queue, "UART QUEUE" , TX_2_ULONG ,
                uart_queue_memory , sizeof(uart_queue_memory));

  /* USER CODE END App_ThreadX_Init */

  return ret;
}

  /**
  * @brief  Function that implements the kernel's initialization.
  * @param  None
  * @retval None
  */
void MX_ThreadX_Init(void)
{
  /* USER CODE BEGIN Before_Kernel_Start */

  /* USER CODE END Before_Kernel_Start */

  tx_kernel_enter();

  /* USER CODE BEGIN Kernel_Start_Error */

  /* USER CODE END Kernel_Start_Error */
}

/* USER CODE BEGIN 1 */

//First Task Reading sensor (high)
void Thread_1_sensor (ULONG thread_input){

      ULONG Task_period = 10; //100Hertz 10ms
      ULONG sonraki_zaman = tx_time_get() + Task_period;

    for(;;){

             counter_1++;
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_SET);

        //MPU6050 ACCEL_XOUT_H (0x3B)
        uint8_t ham_buffer[14];

        HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, MPU6050_REG_ACCEL_XOUT_H,
I2C_MEMADD_SIZE_8BIT, ham_buffer, 14, 100);

        accel_x =  (int16_t)  (ham_buffer[0]<<8) | (ham_buffer[1]);
        accel_y =  (int16_t)  (ham_buffer[2]<<8) | (ham_buffer[3]);
        accel_z =  (int16_t)  (ham_buffer[4]<<8) | (ham_buffer[5]);
        temp_raw = (int16_t)  (ham_buffer[6]<<8) | (ham_buffer[7]);
        gyro_x =   (int16_t)  (ham_buffer[8]<<8) | (ham_buffer[9]);
        gyro_y =   (int16_t)  (ham_buffer[10]<<8)| (ham_buffer[11]);
        gyro_z =   (int16_t)  (ham_buffer[12]<<8)| (ham_buffer[13]);

        ULONG send_data = (ULONG)accel_z;
```

```
        tx_queue_send(&sensor_queue , &send_data , TX_NO_WAIT);

        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_RESET);



           ULONG anlik_zaman = tx_time_get();
           if (sonraki_zaman>anlik_zaman){

              tx_thread_sleep(sonraki_zaman-anlik_zaman);

           }
           sonraki_zaman += Task_period;

           }
}

//Data package (middle)
void Thread_2_data_pack (ULONG thread_input){

       ULONG alinan_veri; //queue den gelen ham veri


       for(;;){

        tx_queue_receive(&sensor_queue,&alinan_veri , TX_WAIT_FOREVER);

              counter_2++;
              HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET);

              int16_t z_ekseni = (int16_t)alinan_veri;
                            //DATA PACKAGE
                            paket.data[0] = 0xAA; //HEADERS
                     paket.data[1] = 0x55;
                     paket.data[2] = (uint8_t)(z_ekseni >> 8); //Z_High
                     paket.data[3] = (uint8_t)(z_ekseni & 0xFF); //Z_low
                     paket.data[4] = '\n'; //Newline

              // Hazırlanan paketi UART queue gönder
              tx_queue_send(&uart_queue, &paket, TX_NO_WAIT);

              HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
               }
}

//Send with UART (LOW)
void Thread_3_UART (ULONG thread_input){

              UART_Packet_t gelen_paket;


              for(;;){
                            tx_queue_receive(&uart_queue , &gelen_paket, TX_WAIT_FOREVER);
                            counter_3++;
                            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_SET);

                            HAL_UART_Transmit(&huart2, gelen_paket.data, 5, 10);

                            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_RESET);
                }
}
```

```
/* USER CODE END 1 */
```